

RESEARCH ARTICLE | MARCH 25 2024

Grad-Shafranov equilibria via data-free physics informed neural networks ^{EP}

Byoungchan Jang  ; Alan A. Kaptanoglu ; Rahul Gaur ; Shaowu Pan ; Matt Landreman ; William Dorland 



Phys. Plasmas 31, 032510 (2024)

<https://doi.org/10.1063/5.0188634>

 CHORUS



Articles You May Be Interested In

A semi-analytical solver for the Grad-Shafranov equation

Phys. Plasmas (November 2014)

A Grad-Shafranov model for compact quasisymmetric stellarators

Phys. Plasmas (April 2025)

Generalized Grad-Shafranov equation for non-axisymmetric MHD equilibria

Phys. Plasmas (October 2020)



Physics of Plasmas

Special Topics Open
for Submissions

[Learn More](#)

Grad-Shafranov equilibria via data-free physics informed neural networks

Cite as: Phys. Plasmas **31**, 032510 (2024); doi: [10.1063/5.0188634](https://doi.org/10.1063/5.0188634)

Submitted: 22 November 2023 · Accepted: 4 March 2024 ·

Published Online: 25 March 2024



View Online



Export Citation



CrossMark

Byoungchan Jang,^{1,a)}  Alan A. Kaptanoglu,²  Rahul Gaur,³  Shaowu Pan,⁴  Matt Landreman,¹ 
and William Dorland¹ 

AFFILIATIONS

¹Institute for Research in Electronics and Applied Physics, University of Maryland, College Park, Maryland 20742, USA

²Courant Institute, New York University, New York, New York 10012, USA

³Department of Mechanical and Aerospace Engineering, Princeton University, Princeton, New Jersey 08544, USA

⁴Department of Mechanical, Aerospace and Nuclear Engineering, Rensselaer Polytechnic Institute, Troy, New York 12180, USA

^{a)} Author to whom correspondence should be addressed: byoungj@umd.edu

ABSTRACT

A large number of magnetohydrodynamic (MHD) equilibrium calculations are often required for uncertainty quantification, optimization, and real-time diagnostic information, making MHD equilibrium codes vital to the field of plasma physics. In this paper, we explore a method for solving the Grad-Shafranov equation by using physics-informed neural networks (PINNs). For PINNs, we optimize neural networks by directly minimizing the residual of the partial differential equation as a loss function. We show that PINNs can accurately and effectively solve the Grad-Shafranov equation with several different boundary conditions, making it more flexible than traditional solvers. This method is flexible as it does not require any mesh and basis choice, thereby streamlining the computational process. We also explore the parameter space by varying the size of the model, the learning rate, and boundary conditions to map various tradeoffs such as between reconstruction error and computational speed. Additionally, we introduce a parameterized PINN framework, expanding the input space to include variables such as pressure, aspect ratio, elongation, and triangularity in order to handle a broader range of plasma scenarios within a single network. Parameterized PINNs could be used in future work to solve inverse problems such as shape optimization.

© 2024 Author(s). All article content, except where otherwise noted, is licensed under a Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC BY-NC-ND) license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>). <https://doi.org/10.1063/5.0188634>

I. INTRODUCTION

The goal of the magnetic confinement fusion research is to establish a hot, dense, steady-state plasma with enough fusion occurring to be economically viable. For fusion parameter regimes, the plasma is often well-described by ideal magnetohydrodynamics (MHD), and 2D equilibria are described by the Grad-Shafranov equation (GSE).^{1,2} For diagnostic and real-time control purpose on current quasi-steady-state fusion devices, accurate plasma equilibria must be computed in real-time from a sparse set of measurement data. These types of calculations can be improved and sped up with neural network surrogate models. Neural network surrogates provide flexibility since the solution is differentiable and meshless. Once these surrogate models are trained, the inference times are very fast and additional improvements in GPUs and neural networks can further improve the performance. Moreover, they can be useful for uncertainty quantification (UQ), as the evaluation of these surrogate models post-training is very inexpensive.

In a physics-informed neural network (PINN), the weights and biases of the network are chosen by directly minimizing the residual of a partial differential equation (PDE) as a loss function.³ Accordingly, PINNs do not need training data obtained through potentially expensive physical experiments or computer simulations.³ PINNs can be used to solve many types of PDEs, including high-dimensional partial differential equations⁴ and fractional partial differential equations.⁵ PINNs can also be used to solve various inverse problems, such as shape optimization,^{6,7} that derive from PDEs.⁸ In this paper, we demonstrate the effectiveness of Physics-Informed Neural Networks (PINNs) in solving the Grad-Shafranov equation, employing both hard boundary conditions for exact enforcement and soft boundary conditions for approximate enforcement through penalty methods, achieving accurate solutions with each variation.

More than two decades ago, neural networks were proposed and briefly studied for solving the ideal MHD force balance equation for tokamaks and stellarators.⁹ However, recently, there has been a

resurgence of interest in neural network surrogates for MHD equilibria.^{10–13} Most of this work has not used physics-informed neural networks (i.e., not directly minimizing the residual of the ideal MHD force balance) or has used substantial experimental data to build a fast surrogate for a specific device. There are also cases where the force balance residual is used as physics regularization¹⁴ or the MHD force balance is solved with a pseudospectral method via a fully spectral polynomial-Fourier basis instead of a typical neural network basis.¹⁵

A. Contributions of this work

In the present work, we generate fully data-free and mesh-free surrogate models of the Grad-Shafranov equation via physics-informed neural networks (PINNs). The intent of this work is not to replace optimized GS solvers but to develop a solver optimized for speed in scenarios where lower accuracy is sufficient, leveraging automatic differentiation (AD) to compute analytic derivatives. This approach eliminates the need for a mesh while requiring the equations to be in a symbolic form, as facilitated by neural networks. Additionally, we introduce a parameterized framework to further broaden the method's applicability, allowing for a diverse range of plasma scenarios to be addressed within a single model. We illustrate the strength of this approach by showing solutions of a class of fixed-boundary tokamak configurations from Cerfon *et al.*, including single and double null configurations.¹⁶ Configurations are implemented with both soft and hard constraints to compare their performance. We also perform hyperparameter scans and other systematic investigations and conclude that the network produces robust results for a very broad range of network parameters. We conclude by demonstrating preliminary work on *parametric* PINNs that are parameterized by the magnitude of the pressure and three shape parameters associated with the “elongated-D” cross section used for tokamak shapes. Unlike standard PINNs in which the models need to be retrained for each boundary shape and choice of profiles, parametric-PINNs only need to be trained once for various configurations. After a modest training period, a parametric-PINN can infer equilibria in tens of milliseconds for different geometries, from negative triangularity tokamaks to field-reversed configurations. For the cases explored here, the solution accuracy is likely sufficient to use a parametric PINN as a surrogate in inverse problems and optimization, although we have not yet explored these applications. Implementing PINNs in this work was done through a modified version of DeepXDE, a Python library for physics-informed learning.¹⁷

II. MHD EQUILIBRIA

MHD equilibria are essential in understanding fusion devices such as tokamaks and stellarators. Equilibrium solutions are used for many applications including diagnostic interpretation, the application of real-time control, and the identification of instability. For toroidal or cylindrical geometries with a symmetry direction, the Grad-Shafranov equation (GSE) defines the equilibria. The GSE is derived from the general ideal-MHD force-balance,

$$\mathbf{J} \times \mathbf{B} = \nabla p. \quad (1)$$

Given a symmetry direction, the divergence-free condition,

$$\nabla \cdot \mathbf{B} = 0, \quad (2)$$

and some vector algebra, we can reduce the force balance equation to the GSE. Details of the derivation can be found in various

textbooks.^{18,19} In short, for an axisymmetric configuration, a poloidal magnetic flux function ψ is introduced, where

$$\mathbf{B} = \frac{1}{r} \nabla \psi \times \mathbf{e}_\phi + \frac{F}{r} \mathbf{e}_\phi \quad (3)$$

$$\mu_0 \mathbf{J} = \frac{1}{r} \frac{dF}{d\psi} \nabla \psi \times \mathbf{e}_\phi - \frac{1}{r} \left[r \frac{\partial}{\partial r} \left(\frac{1}{r} \frac{\partial \psi}{\partial r} \right) + \frac{\partial^2 \psi}{\partial z^2} \right] \mathbf{e}_\phi. \quad (4)$$

Here, ϕ is the toroidal angle, r is the major radial coordinate in cylindrical coordinates, $\mathbf{e}_\phi$ is the unit vector in the toroidal direction, and $F(\psi) = rB_\phi$.

The GSE describes static 2D MHD equilibria via the poloidal flux function $\psi(r, z)$ in standard cylindrical coordinates (r, ϕ, z) as follows:

$$\psi_{rr} - \frac{1}{r} \psi_r + \psi_{zz} + \mu_0 r^2 \frac{dp(\psi)}{d\psi} + \frac{1}{2} \frac{dF^2(\psi)}{d\psi} = 0. \quad (5)$$

Typically, the pressure profiles p and poloidal current profiles $F = rB_\phi$ are free functions that must be prescribed to solve the PDE. In other words, given the pressure and F profiles, solving (5) provides the MHD equilibrium.

A. Solov'ev equilibria

In order to evaluate the performance of PINNs, this work explores a range of analytic GSE solutions.²⁰ Although the general solution for the non-linear partial differential equation in Eq. (5) must be computed numerically, certain choices of $p(\psi)$ and $F(\psi)$ allow for an analytical solution: Solov'ev profiles, in which $p(\psi)$ and $F^2(\psi)$ are both linear functions of ψ

$$\begin{aligned} \mu_0 r^2 \frac{dp(\psi)}{d\psi} &= -Cr^2, \\ \frac{1}{2} \frac{dF^2(\psi)}{d\psi} &= -A, \end{aligned} \quad (6)$$

where A and C are constants. After rescaling with the major radius of the plasma, R_0 , and an arbitrary constant, ψ_0 , via the definitions $R = r/R_0$, $Z = z/R_0$, $\Psi = \psi_0 \Psi$, and $\psi_0 = R_0^2(A + CR_0^2)$ (5) can be written

$$\Psi_{RR} - \frac{1}{R} \Psi_R + \Psi_{ZZ} - P R^2 - (1 - P) = 0, \quad (7)$$

where $P = CR_0^2/(A + CR_0^2)$ is the strength of the linear pressure profile.¹⁶

B. Fixed boundary condition

We consider a toroidal, elongated “D”-shaped geometry with a major radius R_0 , as in Fig. 1. The boundary can be defined by the following parametric equations:²¹

$$\begin{aligned} R &= 1 + \varepsilon \cos(\tau + \arcsin(\delta) \sin(\tau)), \\ Z &= \varepsilon \kappa \sin(\tau), \end{aligned} \quad (8)$$

where $\varepsilon = a/R_0$ is the inverse aspect ratio, κ is the elongation, δ is the triangularity, and $\tau \in [0, 2\pi)$.

In the fixed-boundary scenario, the boundary condition is defined as

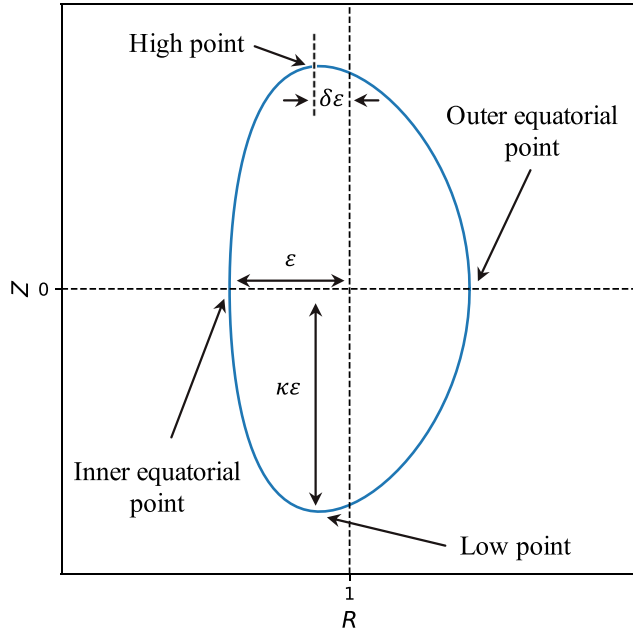


FIG. 1. Elongated-D geometry from Cerfon and Freidberg.¹⁶

$$\Psi(R, Z)|_{\partial\mathcal{D}} = 0, \quad (9)$$

where $\partial\mathcal{D}$ is the boundary from Eq. (8).

C. Analytic solutions for Solov'ev equilibria

In order to find analytic solutions with Solov'ev profiles, we use a method from Cerfon *et al.*¹⁶ With the prescribed boundary conditions and Solov'ev profiles, the Grad-Shafranov equation in Eq. (5) can be solved analytically using a sum of the homogeneous and particular solutions, where

$$\begin{aligned} \Psi &= \Psi_H + \Psi_P, \\ \Psi_P &= \frac{PR^4}{8} + (1-P)\frac{R^2}{2}\log(R), \\ \Psi_H &\approx \sum_{i=1}^7 c_i \Psi_i. \end{aligned} \quad (10)$$

The Ψ_i are the up-down symmetric functions of (R, Z) , and can be found in Cerfon and Friedberg¹⁶ or Zheng *et al.*²² The coefficients c_i are determined from seven point-wise boundary conditions specified in Cerfon *et al.* With the seven geometric constraints, we have seven requirements that uniquely determine $c_1, \dots, c_7$. In principle, for the $\Psi = 0$ contour to coincide with a specific boundary curve such as Eq. (8) exactly, an infinite number of basis functions would need to be included, rather than seven. However, in practice, this truncated analytic solution is very accurate for many choices of the boundary conditions. To be more precise, the relative error between the boundary shape prescribed by Eq. (8) and the boundary shape of the Cerfon analytic solutions is on the order of 10^{-5} for the examples that will be considered here. Given $\nabla\Psi \sim 0.1$ in typical solutions, the difference of

$\sim 10^{-5}$ in the boundary location translates to a difference of $\sim 10^{-5} * 0.1 \approx 10^{-6} = 10^{-4}\%$ in Ψ locally. We conclude that the boundary of the Cerfon analytic solution is close enough to Eq. (8) for Eq. (10) to be used as analytic solutions, and the difference between the boundary shapes is subdominant to other solution errors that will be discussed in the paper.

III. PHYSICS-INFORMED NEURAL NETWORKS

PINNs use a neural network as a universal function approximator²³ to calculate the solution of a PDE given independent variables of the PDE. Using automatic differentiation, residuals at each collocation points are calculated from the governing equation, boundary conditions, and (possibly) data. The residuals are consolidated into a loss function with weights.³ Fig. 3 describes the full PINN architecture used for this work.

Traditionally, feed-forward neural networks (FNNs) are used because of their simplicity and effectiveness; the neurons of adjacent layers are fully connected, and outputs of each layer are fed forward as the inputs to the next layer. FNNs map an input layer $\mathbf{z}_0 \in \mathbb{R}^{n_0}$ to the output $\mathbf{z}_L \in \mathbb{R}^{n_L}$. If $L > 1$ the network is considered “deep.” For the standard PINNs employed in this work, $\mathbf{z}_0 = (R, Z)$, $n_0 = 2$, $n_L = 1$ and $\mathbf{z}_L$ is the neural network approximation of the poloidal flux function, $\bar{\Psi}$. The layers between the input and output layers are denoted hidden layers $\mathbf{z}_i$, $i = 1, \dots, L-1$, and

$$\mathbf{z}_i = \sigma_i(\mathbf{W}_i \mathbf{z}_{i-1} + \mathbf{b}_i), \quad (11)$$

where $\mathbf{W}_i \in \mathbb{R}^{n_i \times n_{i-1}}$ and $\mathbf{b}_i \in \mathbb{R}^{n_i}$ are the weight matrix and the bias vector; the subscript i denotes the index of the layer, and $\sigma_i(\cdot)$ is an activation function acting element-wise. The activation function can be any of the common varieties, e.g., sigmoids, rectified linear units (ReLU), and tanh functions. Different activation functions are explored in this work, and we found that ReLU does not work for solving the Grad-Shafranov equation since the solution needs to be twice differentiable. During training, both PDE and boundary condition residuals in a loss function are calculated by applying automatic differentiation to our differentiable surrogate model $\bar{\Psi}$. After the training, the weights, bias, and activation function at each layer are determined, and the output prediction $\bar{\Psi} = \mathbf{z}_L$ can be efficiently calculated from any given input vector $\mathbf{z}_0$ (i.e., coordinates).

Since we are concerned with the solution to the GSE, we need to specify the equation and an appropriate set of boundary conditions on the boundary $\partial\mathcal{D}$ as follows:

$$\Delta^* \Psi(R, Z) = \mathcal{N}(\Psi, R, Z), \quad (12)$$

$$B(\Psi, R_b, Z_b) = 0, \quad (R_b, Z_b) \in \partial\mathcal{D}, \quad (13)$$

where Δ^* and $\mathcal{N}$ are defined as the following:

$$\Delta^* \Psi(R, Z) = \Psi_{RR} - \frac{1}{R} \Psi_R + \Psi_{ZZ}, \quad (14)$$

$$\mathcal{N}(\Psi, R, Z) = PR^2 + (1-P). \quad (15)$$

$\mathcal{N}$ is for now an arbitrary function of Ψ and the coordinates, determined by the pressure and poloidal current profiles, and $B(\Psi, \mathbf{x})$ denotes the boundary condition for Ψ not a magnetic field. Then, the PINN approach with a “soft” boundary condition can be represented as a minimization problem with a boundary condition as a penalty term as follows:

$$\arg \min_{w,b} \|\Delta^* \bar{\Psi} - \mathcal{N}(\bar{\Psi}, R, Z)\|_2^2 + \lambda_1 \|B(\bar{\Psi}, R_b, Z_b)\|_2^2, \quad (16)$$

where λ_1 is a hyperparameter weighing the trade-off between matching the PDE in the volume and matching the boundary conditions.

For some problems, imposing “hard” instead of soft boundary condition constraints can improve performance.^{24,25} Therefore, both soft constraints and hard constraints for boundary conditions are explored. The hard constraint can be effectively imposed by using a function $G(R, Z)$ that adheres strictly to the boundary condition. The function $G(R, Z)$ can be any smooth function that goes to zero at the boundary. It can be found with two different methods. We can find an analytical form from a given geometry or train a separate neural network that vanishes at the boundary and multiply the two network outputs. Either way, the final neural network with hard constraints, $\bar{\Psi}(R, Z)_{hc}$, becomes

$$\bar{\Psi}(R, Z)_{hc} = G(R, Z) \bar{\Psi}(R, Z). \quad (17)$$

The drawbacks of employing a hard constraint approach include potentially lower accuracy for certain problems and the limitation that it can only be implemented when there is a functional representation of the boundary.

Note that, since the PDE is known, we can generate MHD equilibria without any actual data from a simulation or experiment. Training without any data is advantageous for cases where only a scarce amount of data are available to train neural networks. However, if available, it often improves the results to include a set of data points (R_{data}, Z_{data}) , either numerical or experimental data, where we would like the measured and computed solutions to match,

$$\arg \min_{w,b} \|\Delta^* \bar{\Psi} - \mathcal{N}(\bar{\Psi}, R, Z)\|_2^2 + \lambda_1 \|B(\bar{\Psi}, R_b, Z_b)\|_2^2 + \lambda_2 \|\bar{\Psi}(R_{data}, Z_{data}) - \bar{\Psi}(R_{data}, Z_{data})\|_2^2. \quad (18)$$

$\bar{\Psi}(R_{data}, Z_{data})$ denotes measurements of the poloidal flux function from a simulation or experiment at given points R_{data} and Z_{data} ; in the case of an experiment, $\bar{\Psi}$ should be understood to be an approximation of the GSE corrupted by measurement noise and any non-equilibrium dynamics. Although we solve a number of fixed-boundary problems in Sec. V, we omit any data points to illustrate that these methods can reproduce approximate MHD equilibria from the minimization of the PDE residual and boundary conditions alone. This demonstrates that while PINNs are known to enhance training outcomes when data points are included, for the scenarios examined in this study, utilizing collocation points alone sufficed to achieve a training accuracy of 1×10^{-4} . Unlike data points that are from either a simulation or experiment, collocation points are points in input space where the residuals are calculated. Collocation points are different from data points in that there is no need for experiment or simulation to collect and label the data. An example of collocation points used in this work is shown in Fig. 2.

IV. IMPLEMENTATION AND TRAINING

The general architecture used for this work is shown in Fig. 3. We will first use a standard FNN with three hidden layers and 20 nodes in each hidden layer. The inputs are the coordinates (R, Z) , and the output is $\bar{\Psi}$. To try to avoid local optima, the Adam optimizer²⁶ with learning rate $\alpha_l = 10^{-3}$ was used for the first 100 iterations and then

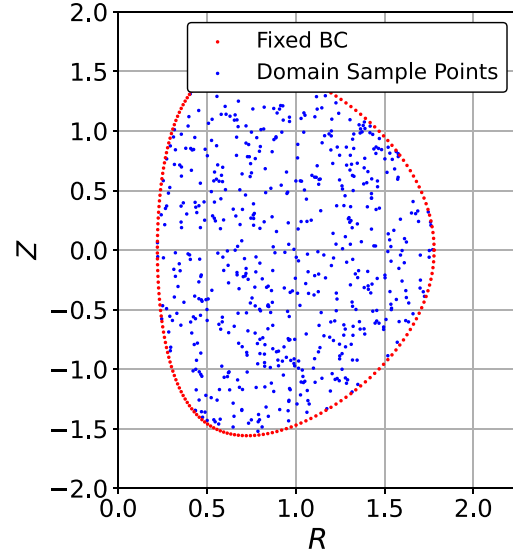


FIG. 2. 500 collocation points inside the domain where a governing equation should be satisfied and 100 collocation points on the boundary are plotted.

L-BFGS-B²⁷ is used. We used 100 and 500 boundary and domain collocation points, respectively. In Appendix, the parameter space is explored by varying the size of the model, activation functions, and optimizers, in order to map various tradeoffs such as between reconstruction error and computational speed. We demonstrate that approximating the solution to the two-dimensional Grad-Shafranov equation with linear profiles can be done with absolute errors of no more than one percent. We observe no significant effect on our results when using larger networks and adjusting the hyperparameters. In the subsections that follow, we will explain how we use PINN in various scenarios.

A. Simple equilibria

The first equilibria considered have smooth cross sections described by Eq. (8). The domain collocation points are randomly generated using Latin hypercube sampling. These points are where the residual of the GS PDE (7) is calculated. To be clear, these collocation points and residuals are not obtained via running other equilibrium solvers. The collocation points are merely points where the residuals will be calculated. Equation (8) is used to apply a Dirichlet boundary condition (i.e., $\Psi_b = 0$) along the cross-section boundary. The residuals of the boundary loss will be calculated according to Eq. (21). The calculated residuals are then used to calculate the loss function

$$L_{total} = L_{pde} + \lambda_1 L_{bc}, \quad (19)$$

$$L_{pde} = \frac{1}{N_{pde}} \sum_{j=1}^{N_{pde}} \left\| \Psi_{RR} - \frac{1}{R} \Psi_R + \Psi_{ZZ} - PR^2 - (1 - P) \right\|_2^2, \quad (20)$$

$$L_{bc} = \frac{1}{N_{bc}} \sum_{j=1}^{N_{bc}} \|\Psi(R_j, Z_j)\|_2^2. \quad (21)$$

We found that the highest accuracy is obtained with $\lambda_1 = 100$ for all the cases explored in this work.

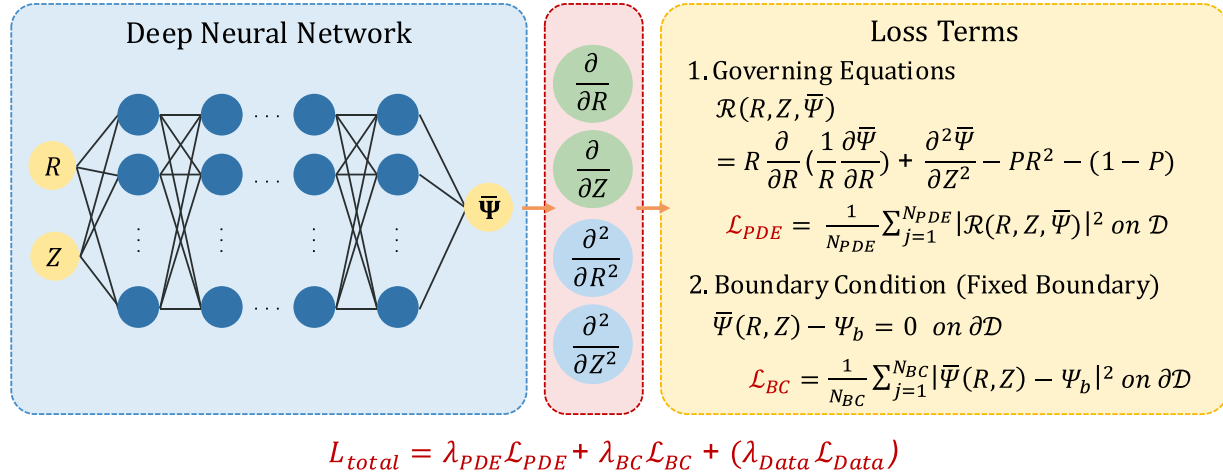


FIG. 3. PINN Architecture for the Grad-Shafranov equation with Solov'ev profiles used for this work. The input space R, Z are the collocation points where the residuals of Grad-Shafranov equation are calculated. The residuals are calculated through automatic differentiation as represented in the red bubble.

B. Equilibria with a divertor/X-point

Equilibria with an X-point can be calculated similar to the simple equilibria examples. Given the absence of a convenient parametric equation for the boundary, akin to Eq. (8), we derive the boundary through an analytical solution as described in Sec. II C. We can then generate collocation points both in the domain and at the boundary.

C. Hard constraints

In the method shown in Fig. 3, we apply soft constraints of the boundary conditions through the loss L_{bc} shown in Eq. (21). Soft constraints on the boundary condition were used for simple- and divertor-equilibria cases. The hard constraints on the boundary conditions can be advantageous for various reasons. For some problems, it is shown that the hard constraints produce faster and more accurate solutions to a given PDE. It might be important in various diagnostic scenarios or in shape optimization to match the boundary condition exactly.

For this work, we derived the following hard constraint function $G(R, Z)$ from Eq. (8):

$$G(R, Z) = |Z| - \varepsilon \kappa \sin \left(\arccos \left(\frac{|R| - 1}{\varepsilon} \right) - \arcsin(\delta) \frac{|Z|}{\varepsilon \kappa} \right). \quad (22)$$

D. Parametric-PINN

So far, each trained PINN model had fixed shape parameters, so the PINN can only produce solutions for one specific boundary configuration. However, it can be helpful to be able to interpolate and extrapolate for various boundary configurations with one pre-trained model. This can allow us to solve inverse problems, such as shape optimization for reactor design. When expanding to the parametric-PINN framework, the input space includes additional variables: strength of the pressure profile P , inverse aspect ratio ε , elongation κ , and triangularity δ . The expanded input space allows PINNs to vary the boundary condition loss term from Eq. (13). This extended input space requires modifications to the training and implementation procedures, as depicted in Fig. 4. Instead of only the spatial (R, Z) coordinates, the

input to the PINN is now a six-dimensional vector $[R, Z, P, \varepsilon, \kappa, \delta]$ and the neural network output is $\bar{\Psi}(R, Z, P, \varepsilon, \kappa, \delta)$. In principle, one could include other parameters that affect the equilibria as additional variables in the input layer.

For the PDE residuals, we use the Grad-Shafranov equation with varying P . Soft boundary conditions are used, in the same way as in the non-parametric case. The implementation of the trained parametric-PINN model can be realized by feeding it with the required six-dimensional inputs, thereby facilitating accurate and efficient solutions to the Grad-Shafranov equation under different plasma scenarios characterized by varied pressure profiles, aspect ratios, elongations, and triangularities.

V. RESULTS

We first demonstrate the accuracy and effectiveness of PINNs for solving the GSE under two different Solov'ev profiles for four different devices: International Thermonuclear Experimental Reactor (ITER),

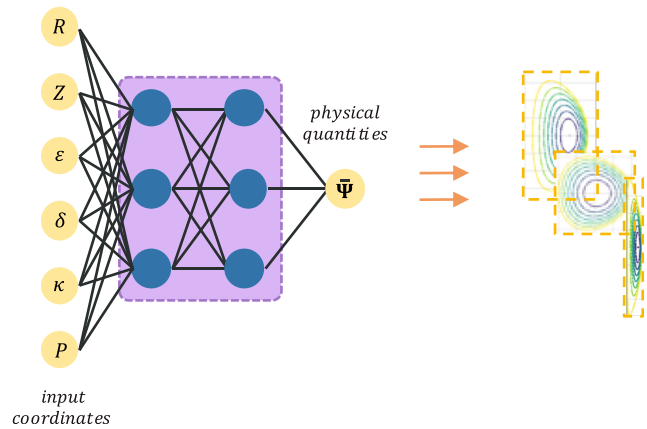


FIG. 4. Modified input space for parametric PINN. One trained parametric-PINN can generate equilibria for different system configurations.

National Spherical Torus Experiment (NSTX), a spheromak, and an FRC (field-reversed configuration). In addition to these configurations, we also investigated variations with divertors, further expanding the applicability of our method. We benchmark the PINN-based solutions against the analytical solutions in Eq. (10), which serve as a reliable reference for our study.

In addition, we examine the impact of using hard constraints instead of soft constraints in the PINN model. Our findings show that hard constraints decrease computational complexity and relative error, as they satisfy an analytical representation for the boundary by definition, eliminating the need to evaluate collocation points for each boundary.

Finally, we present preliminary results exploring parametric PINN, where a single neural network can be trained for various devices with different geometries. Our model suffered from the curse of dimensionality, as the model's larger input space was associated with an exponentially increased number of boundary collocation points. Nevertheless, we were able to train a reasonably effective surrogate for MHD equilibria parameterized by the shape parameters, with promise for inverse problems.

A. Baseline equilibria with different configurations

The seven different configurations considered in this study are displayed in Table I in Appendix. By assessing the performance of PINNs in these diverse scenarios, our objective is to demonstrate the robustness and versatility of our approach to solving GSE in various magnetic confinement devices. As one example, the NSTX geometry with finite beta is shown in Fig. 5.

We evaluate the performance of PINNs by analyzing several important figures of merit, such as relative error (both average and maximum) and computational time (both training and inference time). These metrics provide insight into the accuracy, efficiency, and overall effectiveness of our approach to modeling the different plasma configurations. Our main metric for performance is the relative error as a percent

$$E(R, Z) = 100\% \frac{|\bar{\Psi} - \Psi^*|}{|\bar{\Psi}_a|}, \quad (23)$$

where $\bar{\Psi}$ is a neural network surrogate for the flux function, Ψ^* is the analytic solution, and $\bar{\Psi}_a$ is the maximum value of the neural network

TABLE I. Shape parameters and pressure for all the configurations explored in this work. Here, P is the pressure parameter in Eq. (7), $\varepsilon = a/R_0$ is the inverse aspect ratio, κ is the elongation, and $\sin(\alpha) = \delta$ is the triangularity.

Configuration				
Devices	P	ε	κ	δ
ITER	0.0	0.32	1.7	0.33
NSTX	0.0	0.78	2.0	0.35
Spheromak	0.0	0.95	1.0	0.2
ITER	1.0	0.32	1.7	0.33
NSTX	1.0	0.78	2.0	0.35
Spheromak	1.0	0.95	1.0	0.2
FRC	1.0	0.99	10.0	0.7

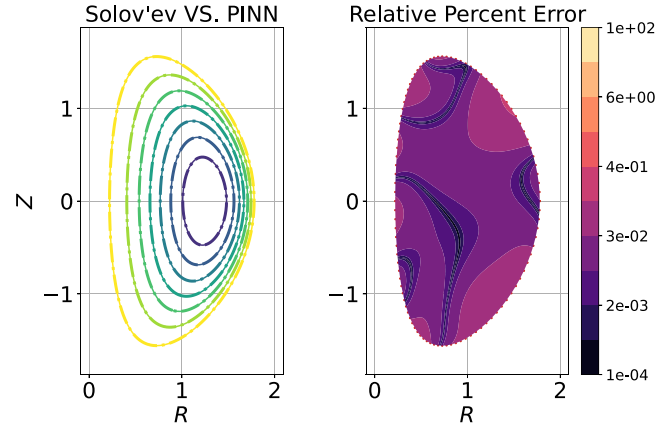


FIG. 5. MHD equilibrium calculated using NSTX geometry with finite beta. The plot on the left shows the solid and dot-dashed curves that indicate the PINN and analytic solutions, respectively.

surrogate for the flux function, i.e., the value at the magnetic axis. Table II shows the ability of our method to accurately and effectively solve the GSE for a wide range of plasma geometries and magnetic field conditions. The spatially averaged relative errors were around 0.02%, which is low enough to closely reproduce the correct equilibrium. Notably, increasing the number of layers or nodes in the neural network only marginally reduces these errors further, unlike traditional equilibrium solvers, which can find equilibria to high numerical precision. This loss of accuracy compared to traditional numerical solvers is inevitable,²⁹ so PINNs should be used for problems where computational speed or inverse solves are more important than obtaining equilibria to high numerical precision.

B. Configurations with a divertor

For a diverted configuration with X-point(s) in the separatrix, the same neural network architecture is used, with some modification in the boundary conditions. Similar to the simple equilibria case, we implemented these configurations with a fixed boundary condition. Since we do not have an explicit analytic expression for the boundary shape, we numerically located points on the boundary of the analytic

TABLE II. Performance of the PINNs for the baseline configurations explored in this work. Typical reconstruction time for a fast-solver like rt-EFIT would be anywhere from 2 ~ 50 ms.²⁸

Configuration		Average error (%)	Max error (%)	Training time (s)	Inference time (ms)
Devices	P				
ITER	0	0.015	0.162	20	10
NSTX	0	0.032	0.142	22	15
Spheromak	0	0.027	0.204	36	12
ITER	1	0.021	0.136	17	11
NSTX	1	0.026	0.186	32	13
Spheromak	1	0.075	0.325	37	10
FRC	1	0.057	0.214	27	17

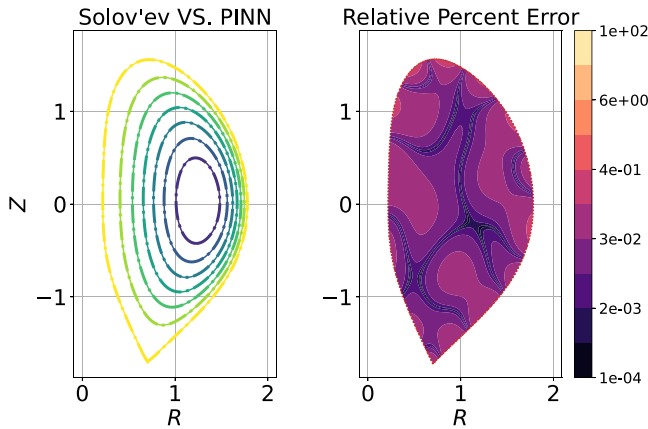


FIG. 6. MHD equilibrium calculated using NSTX geometry with finite beta with a lower divertor.

solution for Ψ and used the resulting points for the boundary loss term. As shown in Fig. 6, the accuracy and training time were comparable to the baseline equilibria configurations without an X-point.

C. Hard vs soft constraints

PINNs for configurations without an X-point were trained using both hard and soft constraints, and the figures of merit mentioned earlier were compared: equilibrium reconstruction error (both average and maximum) and computational time (both training and inference time). The results with hard constraints show 2–10 times more accurate equilibria reconstructions with faster training time. Qualitatively, we can see that the error at the boundary is zero by Eq. (17) and can be visually confirmed in Fig. 7. As mentioned before, even though the Cerfon analytic solution does not align exactly with Eq. (8), using the Cerfon analytic solution as a comparison is justified as the relative error between the two is subdominant, i.e., $\sim 10^{-4}\%$.

D. Parametric-PINNs

We present a preliminary work on parametric-PINNs in which a single neural network is trained for various devices with different geometries. A single trained model can interpolate and even extrapolate for various boundary configurations. For training, we varied each parameter (e.g., P , ϵ , κ , and δ) as follows: $P = [0.00, 1.00]$, $\epsilon = [0.12, 0.52]$, $\kappa = [1.25, 2.75]$, and $\delta = [-0.5, 0.5]$. For each parameter, we sampled nine points in the given range. The model had four layers with 30 nodes in each layer.

The training for the parametric-PINNs took longer than the regular PINNs (i.e., four hours vs 20 seconds) and the reconstruction error increased by up to an order of magnitude as shown in Fig. 8. The inference times are comparable to regular PINNs at around 10 ms. With 6D input instead of 2D input space for the model, the number of collocation points exponentially increased. The number of collocation points scales as N^D where D is the number of dimensions and N is the number of points sampled in each dimension (assuming for simplicity one does uniform sampling). For example, we have four dimensions (i.e., P , ϵ , κ , δ) with nine points sampled in each dimension (i.e., for triangularity: $-0.5, -0.375, -0.25, -0.125, 0.0, 0.125, 0.25, 0.375$, and

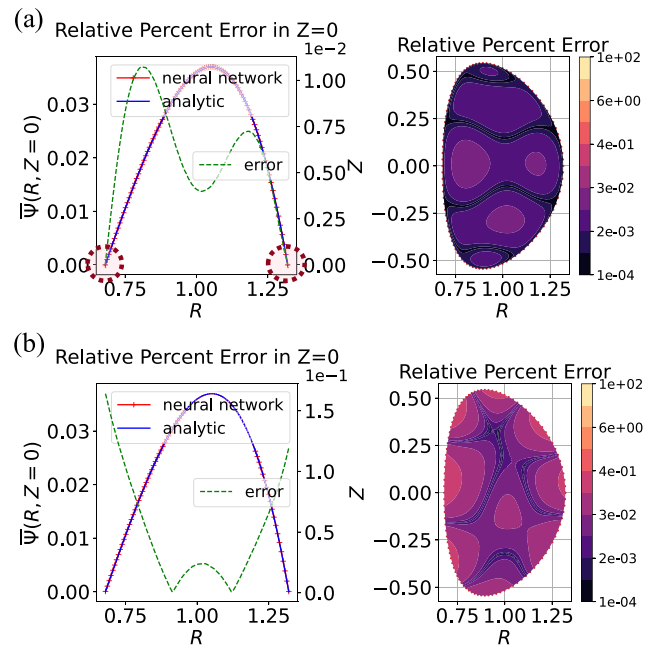


FIG. 7. Comparing soft and hard constraints for ITER configuration. The model shown in panel (a) is a hard constraint model and in panel (b) is a soft constraint model. For the hard constraint case (a), the error is zero on the boundary, verifying the hard constraint.

0.5). So, if we have a 100 boundary collocation points (R, Z) for each configuration (i.e., at a fixed P , ϵ , κ , δ), the total number of boundary collocation points for parametric-PINNs will be $100 \times 9^4 = 656\,100$. For the domain collocation points, we also used 656 100 pseudo-randomly generated points.

A preliminary result on the trained parametric-PINNs is shown in Fig. 8, which shows the variation in δ from a single trained model instead of three different models. The three other dimensions (i.e., P , ϵ ,

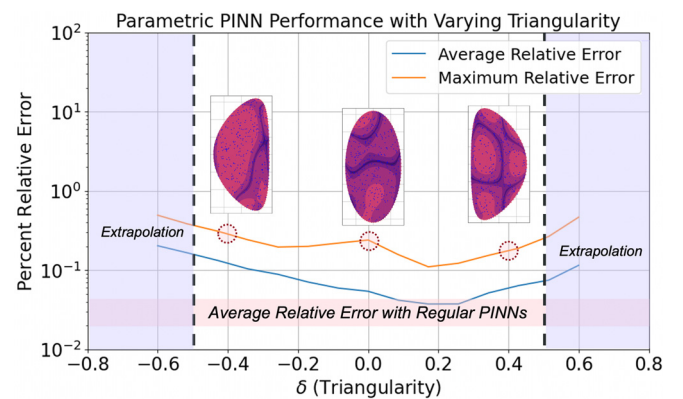


FIG. 8. Relative percent error of a pretrained parametric-PINN for various triangularities. At the worst interpolation regions, it is an order of magnitude worse than a regular PINN. The colorbars of the configurations are the same as the colorbars in Figs. 5–7.

and δ) can also be varied in the same model without re-training, but results are not shown here as the trend is similar. The error gradually gets worse as we extrapolate further from the trained region.

Parametric PINNs have the potential to be as accurate as regular PINNs with further hyperparameter tuning and other techniques such as hard constraints and adaptive sampling methods.^{30,31} Exploring hard constraints for boundary conditions in parametric PINNs is beyond the scope of this work, but hard constraints could be advantageous by providing a workaround for the exponentially increasing number of collocation points. Furthermore, given the high-dimensional input space, strategies for efficient sampling of collocation points (both interior and boundary points) are also imperative in future work.

VI. SUMMARY AND CONCLUSIONS

In this study, we have illustrated a comparison between the analytical solution and PINN in various fusion devices, such as ITER, NSTX, a spheromak, and an FRC. Although the PINNs approach may not be the optimal choice for achieving the highest precision compared to modern numerical MHD equilibrium codes, it offers several advantages. Specifically, the PINN approach (1) eliminates the need for complicated PDE discretization and solution schemes, requiring only the coding of the PDE in symbolic language and (2) provides flexibility through automatic differentiation, enabling its use on various geometries and plasma devices without mesh generation.

The potential of PINNs to incorporate physical constraints into the loss function, as well as its flexibility, implies that it could be a valuable tool for MHD equilibrium reconstruction and optimization in a variety of devices with further improvements. PINNs are also known to excel in addressing inverse problems, a topic that should be explored in future work.

Our work makes initial progress in employing PINNs for plasma physics applications, but we acknowledge that there is still room for improvement and further exploration. Future research should concentrate on refining the model by improving sampling methods for collocation points and broadening its applicability to a wider range of magnetic confinement devices. Although our preliminary parametric PINN approach encounters obstacles such as the curse of dimensionality, future work can investigate potential solutions, including separable PINNs,³² hard constraints, and adaptive sampling techniques.^{30,31} This work can also expand to introduce general pressure and current profiles instead of the linear profiles shown in this work. Then, this work can expand to free-boundary equilibria, shape optimization for tokamaks, and 3D equilibria for stellarators, which have more complicated features such as magnetic islands.

ACKNOWLEDGMENTS

We would like to acknowledge valuable discussions with Chris Hansen, Andrea Merlo, Timo Thun, and Thomas Sunn Pederson. This work was supported by the U.S. Department of Energy under Contract No. DE-FG02-93ER54197. This research used resources of the National Energy Research Scientific Computing Center (NERSC), a U.S. Department of Energy Office of Science User Facility located at the Lawrence Berkeley National Laboratory, operated under Contract No. DE-AC02-05CH11231 using NERSC award FES-ERCAP-mp217-2023.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Byoungchan Jang: Conceptualization (equal); Data curation (lead); Formal analysis (lead); Investigation (lead); Methodology (lead); Software (lead); Validation (lead); Visualization (lead); Writing—original draft (lead). **Alan A. Kaptanoglu:** Conceptualization (equal); Data curation (supporting); Investigation (supporting); Methodology (supporting); Project administration (equal); Software (equal); Supervision (equal); Writing—review & editing (equal). **Rahul Gaur:** Investigation (supporting); Methodology (supporting); Writing—review & editing (supporting). **Shaowu Pan:** Conceptualization (supporting); Writing—review & editing (supporting). **Matt Landreman:** Conceptualization (equal); Funding acquisition (equal); Project administration (equal); Supervision (equal); Writing—review & editing (equal). **William Dorland:** Funding acquisition (equal); Supervision (equal); Writing—review & editing (equal).

DATA AVAILABILITY

The data that support the findings of this study are available from the corresponding author upon reasonable request.

APPENDIX: HYPERPARAMETER ANALYSIS

The parameter space of the PINN architecture is explored by varying the activation functions, learning rate, number of iterations of Adam optimizer before using L-BFGS-B, and the size of the model, in order to map various tradeoffs such as between reconstruction error and computational speed. For each set of hyperparameters, we repeated the training ten times with different random seeds for initialization.

For activation functions, there are four that were explored: sigmoid, swish, tanh, and ReLU:

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad (\text{A1})$$

$$\text{swish}(x) = x\sigma(x), \quad (\text{A2})$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad (\text{A3})$$

$$\text{ReLU}(x) = \max(0, x). \quad (\text{A4})$$

The sigmoid function is used as an activation function in binary classification problems, yielding a value between zero and one, which can be interpreted as a probability. However, sigmoid can suffer from the vanishing gradient problem, which can make training of deep networks challenging.^{33,34}

The hyperbolic tangent (tanh) function returns values that range from -1 to 1 . The tanh function is similar to the sigmoid function but the mean of its output is closer to zero, which helps with converging the model during gradient descent.³³

The rectified linear unit (ReLU) returns the input directly if it is positive; otherwise, it evaluates to zero.^{35,36} ReLU has become a default choice for many problems due to its simplicity and

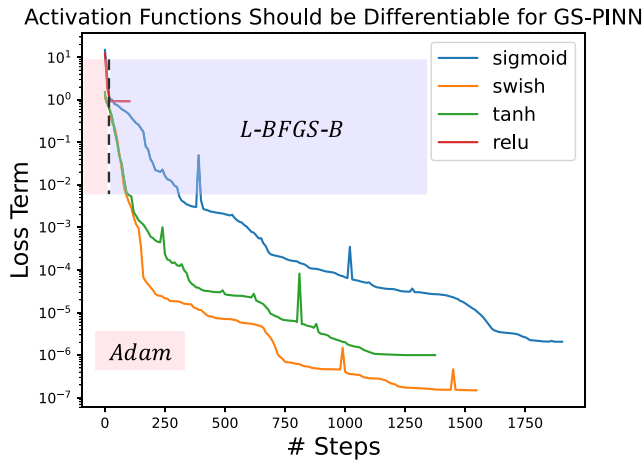


FIG. 9. For these runs, ten Adam steps were used and the swish function shows the best performance. ReLU fails to converge for all variations.

performance across multiple datasets and tasks. Its major benefit is that it solves the vanishing gradient problem, allowing models to learn faster and perform better. The issue with ReLU for this work is that the function is not differentiable at the origin. It is clearly shown in Fig. 9 that models with ReLU do not converge.

Swish attempts to combine the best properties of ReLU and sigmoid functions. It is non-monotonic, and this can help the model learn more complex patterns. It is smooth and differentiable, and unlike ReLU, it does not nullify negative input values.³⁷

PINNs were trained with these four activation functions with a learning rate of 2×10^{-2} , with 100 Adam iterations followed by L-BFGS-B, network size of three hidden layers with 20 nodes each, 100 boundary points, and 1000 domain points. As shown in Fig. 9, swish and tanh perform significantly better than the sigmoid function, and it is inconclusive whether tanh or swish is optimal. Models had a hard time converging with ReLU. This is probably because for the Grad-Shafranov equation, the model needs to be two times

Optimal Learning Rates is 2e-2 for 5000 adam steps

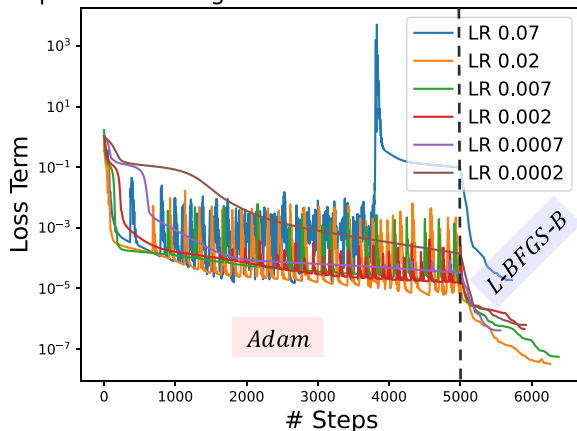


FIG. 10. Learning rate of Adam optimizer of anything higher than 2×10^{-2} starts to diverge due to taking big steps.

Running 10 Adam Steps Produces the Best Result

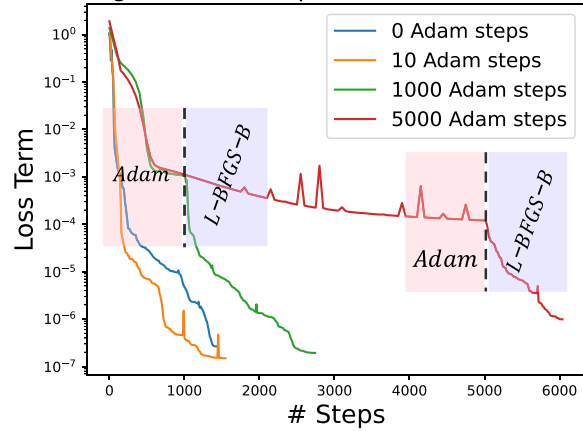


FIG. 11. Running a smaller number of Adam optimizer before L-BFGS-B optimizer generally achieves higher accuracy and shorter computation time. However, fewer Adam steps cause greater uncertainties in the end result depending on the initialization.

differentiable, where the ReLU function is not differentiable at the origin.

Next, the learning rate was explored by varying it from 7×10^{-1} to 2×10^{-4} . The learning rate of the Adam optimizer is considered an important hyperparameter for the effective neural network training.³⁸ Depending on each model's loss landscape, an optimized learning rate can provide an effective training. If the learning rate is too high, the model may diverge, whereas if the learning rate is too low, it may take too long to converge. As shown in Fig. 10, for 1000 Adam steps, the optimal learning rate is around 2×10^{-2} and we see that the model starts to diverge at 7×10^{-2} . Note that the choice of these learning rates is arbitrary.

Next, we vary the number of training steps from the Adam optimizer before L-BFGS-B is applied. PINNs are typically trained with two optimizers in succession: Adam and then L-BFGS-B.^{29,39} Since L-BFGS-B is a second-order optimization method, it converges to minima more accurately. However, L-BFGS-B can rapidly

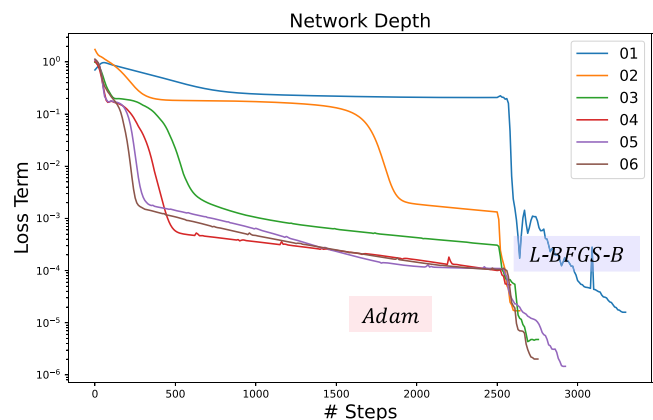


FIG. 12. There is no significant improvement in accuracy for anything deeper than three layers.

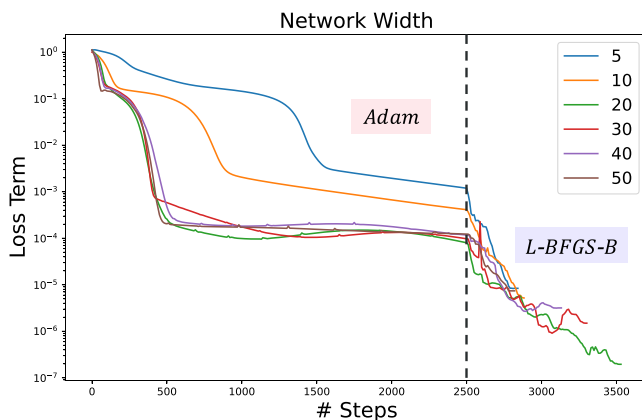


FIG. 13. There is no significant improvement in accuracy for anything more than 20 nodes wide.

converge and get stuck to local minima. This is why we use Adam before L-BFGS-B to avoid local minima. In Fig. 11, our runs show that Adam is helpful for robust training, since a low number of Adam training steps results in high variation depending on the initialization.

Next, we vary the size of the network, first the depth of the network (i.e., the number of hidden layers) and then the width of the network (i.e., the number of nodes for each hidden layer). We find that anything more than three hidden layers does not show a significant increase in the performance of the model as shown in Fig. 12. For the width, anything more than 20 nodes does not show a significant increase in the performance as shown in Fig. 13. This is probably due to the smooth and relatively well-behaved nature of the Grad-Shafranov solutions with linear profiles.

REFERENCES

- ¹H. Grad and H. Rubin, "Hydromagnetic equilibria and force-free fields," *J. Nucl. Energy* (1954) **7**, 284–285 (1958).
- ²V. Shafranov, "On magnetohydrodynamical equilibrium configurations," *Sov. Phys. JETP* **6**, 1013 (1958).
- ³M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.* **378**, 686–707 (2019).
- ⁴J. Blechschmidt and O. G. Ernst, "Three ways to solve partial differential equations with neural networks—a review," *GAMM-Mitteilungen* **44**, e202100006 (2021).
- ⁵G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nat. Rev. Phys.* **3**, 422–440 (2021).
- ⁶K. Shukla, V. Oommen, A. Peyvan, M. Penwarden, L. Bravo, A. Ghoshal, R. M. Kirby, and G. E. Karniadakis, "Deep neural operators can serve as accurate surrogates for shape optimization: A case study for airfoils," *arXiv:2302.00807* (2023).
- ⁷R. Rossi, M. Gelfusa, and A. Murari, "On the potential of physics-informed neural networks to solve inverse problems in tokamaks," *Nucl. Fusion* **63**, 126059 (2023).
- ⁸S. Cai, Z. Wang, S. Wang, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks for heat transfer problems," *J. Heat Transfer* **143**, 060801 (2021).
- ⁹B. P. van Milligen, V. Tribaldos, and J. Jiménez, "Neural network differential equation and plasma equilibrium solver," *Phys. Rev. Lett.* **75**, 3594 (1995).
- ¹⁰S. Joung, J. Kim, S. Kwak, J. Bak, S. Lee, H. Han, H. Kim, G. Lee, D. Kwon, and Y.-C. Ghim, "Deep neural network Grad-Shafranov solver constrained with measured magnetic signals," *Nucl. Fusion* **60**, 016034 (2019).
- ¹¹A. Merlo, D. Böckenhoff, J. Schilling, U. Höfel, S. Kwak, J. Svensson, A. Pavone, S. A. Lazerson, and T. S. Pedersen, and the W7-X Team, "Proof of concept of a fast surrogate model of the VMEC code via neural networks in Wendelstein 7-X scenarios," *Nucl. Fusion* **61**, 096039 (2021).
- ¹²J. Wai, M. Boyer, and E. Kolemen, "Neural net modeling of equilibria in NSTX-U," *arXiv:2202.13915* (2022).
- ¹³Y. Liu, C. Akcay, L. L. Lao, and X. Sun, "Surrogate models for plasma displacement and current in 3-D perturbed magnetohydrodynamic equilibria in tokamaks," *Nucl. Fusion* **62**, 126067 (2022).
- ¹⁴A. Merlo, D. Böckenhoff, J. Schilling, S. A. Lazerson, and T. S. Pedersen, and the W7-X Team, "Physics-regularized neural network of the ideal-MHD solution operator in Wendelstein 7-X configurations," *Nucl. Fusion* **63**, 066020 (2023).
- ¹⁵D. Dudt and E. Kolemen, "DESC: A stellarator equilibrium solver," *Phys. Plasmas* **27**, 102513 (2020).
- ¹⁶A. J. Cerfon and J. P. Freidberg, "'One size fits all' analytic solutions to the Grad-Shafranov equation," *Phys. Plasmas* **17**, 032502 (2010).
- ¹⁷L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, "DeepXDE: A deep learning library for solving differential equations," *SIAM Rev.* **63**, 208–228 (2021).
- ¹⁸J. P. Freidberg, *Ideal MHD* (Cambridge University Press, 2014).
- ¹⁹R. D. Hazeltine and J. D. Meiss, *Plasma Confinement* (Courier Corporation, 2003).
- ²⁰L. Solov'ev, "The theory of hydromagnetic stability of toroidal plasma configurations," *Sov. Phys. JETP* **26**, 400–407 (1968).
- ²¹R. Miller, M. Chu, J. Greene, Y. Lin-Liu, and R. Waltz, "Noncircular, finite aspect ratio, local equilibrium model," *Phys. Plasmas* **5**, 973–978 (1998).
- ²²W. Zheng, F. Hu, M. Zhang, Z. Chen, X. Zhao, X. Wang, P. Shi, X. Zhang, X. Zhang, Y. Zhou, Y. Wei, and Y. Pan, and J-TEXT team, "Hybrid neural network for density limit disruption prediction and avoidance on J-TEXT tokamak," *Nucl. Fusion* **58**, 056016 (2018).
- ²³K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks* **2**, 359–366 (1989).
- ²⁴L. Sun, H. Gao, S. Pan, and J.-X. Wang, "Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data," *Comput. Methods Appl. Mech. Eng.* **361**, 112732 (2020).
- ²⁵L. Lu, R. Pestourie, W. Yao, Z. Wang, F. Verdugo, and S. G. Johnson, "Physics-informed neural networks with hard constraints for inverse design," *arXiv:2102.04626* (2021).
- ²⁶D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv:1412.6980* (2014).
- ²⁷C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal, "Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization," *ACM Trans. Math. Software* **23**, 550–560 (1997).
- ²⁸J. Ferron, M. Walker, L. Lao, H. S. John, D. Humphreys, and J. Leuer, "Real time equilibrium reconstruction for tokamak discharge control," *Nucl. Fusion* **38**, 1055 (1998).
- ²⁹S. Markidis, "The old and the new: Can physics-informed deep-learning replace traditional linear solvers?," *Front. Big Data* **4**, 669097 (2021).
- ³⁰K. Tang, X. Wan, and C. Yang, "DAS-PINNS: A deep adaptive sampling method for solving high-dimensional partial differential equations," *J. Comput. Phys.* **476**, 111868 (2023).
- ³¹C. Zhu, M. Zhu, Q. Tan, Y. Kartha, and L. Lu, "A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks," *Comput. Methods Appl. Mech. Eng.* **403**, 115671 (2023).
- ³²J. Cho, S. Nam, H. Yang, S.-B. Yun, Y. Hong, and E. Park, "Separable PINN: Mitigating the curse of dimensionality in physics-informed neural networks," *arXiv:2211.08761* (2022).
- ³³Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient backprop," in *Neural Networks: Tricks of the Trade* (Springer, 2002), pp. 9–50.

- ³⁴S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM J. Sci. Comput.* **43**, A3055–A3081 (2021).
- ³⁵V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in Proceedings of the 27th International Conference on Machine Learning (ICML-10) (ICML, 2010), pp. 807–814.
- ³⁶A. F. Agarap, "Deep learning using rectified linear units (ReLU)," [arXiv:1803.08375](https://arxiv.org/abs/1803.08375) (2018).
- ³⁷P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," [arXiv:1710.05941](https://arxiv.org/abs/1710.05941) (2017).
- ³⁸Y. Wu, L. Liu, J. Bae, K.-H. Chow, A. Iyengar, C. Pu, W. Wei, L. Yu, and Q. Zhang, "Demystifying learning rate policies for high accuracy training of deep neural networks," in *2019 IEEE International Conference on Big Data (Big Data)* (IEEE, 2019), pp. 1971–1980.
- ³⁹D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Math. Program.* **45**, 503–528 (1989).